

Smali By bithowl: Chapter 11 Register Instructions

“Follow the Data”



Move

Create

Access

Invoke

Return

...

SMALI BY BITHOWL : CHAPTER 11

REGISTER INSTRUCTIONS

“Follow the Data”

In the previous chapter, you learned what can live inside a register. Now we learn how data actually moves.

THE BIG PICTURE

Data → Register → Instruction
Register → Method → ...

Follow the data, not just the instruction.



8. THE STORY (HOOK)

The Warehouse Conveyor Belt

Think of registers as numbered boxes in a warehouse. Workers move boxes around using specific instructions.



- Boxes = Registers
- Workers = Instructions
- Contents = Values (int, string, object, etc.)

The box doesn't tell you what's inside. You need to follow how it's used.

THE TWO REGISTER FAMILIES

v REGISTERS (Local / Working)

v0 v1 v2 ...

- Temporary values
- Local variables
- Results
- Intermediate data

p REGISTERS (Parameters)

p0 p1 p2 ...

- Method parameters
- p0 = this (instance methods)
- First argument (static methods)

They are not separate register sets. They are different names for parts of the same register frame.

3. REGISTER TYPES – THE BIG PICTURE

Type Category	Java Type(s)	Small Descriptor(s)	Registers
Single-width (1 register)	int	I	1
	boolean	Z	1
	byte	B	1
	char	C	1
	short	S	1
	float	F	1
Wide (2 registers)	long	J	2
	double	D	2
References (1 register)	object / array / string	L...; / [...] ;	1

Object and array values are references, not the actual data.

1. MOVE

Copy a value from one register to another.

```
const/4 v0, 0x2A  
move v1, v0
```

Before

After

v0 = 42

v1 = ?

v0 = 42

v1 = 42

move copies the value. It does not clear the source register.

2. MOVE-OBJECT

Copy an object/reference.

```
const-string v0, "hello"  
move-object v1, v0
```

v0 → "hello"

v1 → "hello"

Moves a reference, not the full object.

3. MOVE-WIDE

Move 64-bit values (long/double).

```
const-wide v0, 0x1122334455667788  
move-wide v2, v0
```

v0 + v1 → long

v2 + v3 → same long

Wide values occupy two consecutive registers.

4. MOVE-RESULT / MOVE-RESULT-WIDE

move-result v0 (int/boolean/etc.)
move-result-object v0 (object/reference)
move-result-wide v0 (long/double)

method() → result → move-result → register

5. CONST & CONST-STRING

Load constants and string values.

```
const/4 v0, 0x1  
const-string v1, "admin"
```

v0 = 1 (int)

v1 = "admin" (string reference)

Strings are objects, so they use object references.

6. NEW-INSTANCE

Create a new object.

```
new-instance v0, Lcom/example/User;  
invoke-direct (v0), Lcom/example/User;.<init>()V
```

v0 → new User()

Usually followed by a constructor call. (invoke-direct).

7. IGET / IPUT

Read or write instance fields.

```
iget-object v0, p0, Lcom/example/User;.>token:Ljava/lang/String;  
sput-object v0, p0, Lcom/example/User;.>token:Ljava/lang/String;
```

iget (read) sput (write)

Works with all field types (int, String, object, etc.).

8. SGET / SPUT

Read or write static fields.

```
sget-object v0, Lcom/example/Config;.>API_URL:Ljava/lang/String;  
sput-object v0, Lcom/example/Config;.>API_URL:Ljava/lang/String;
```

sget (read) sput (write)

Static fields belong to the class, not an object.

9. REGISTER TYPES QUICK REFERENCE

Java Type	Descriptor	Width
boolean	Z	1
byte	B	1
char	C	1
short	S	1
int	I	1
float	F	1
long	J	2
double	D	2
void	V	—
String	Ljava/lang/String;	1 (ref)
Object	L...;	1 (ref)
int[]	[I	1 (ref)
String[]	[Ljava/lang/String;	1 (ref)

1. PARAMETERS & RETURN VALUES

Instance Method

p0 = this
p1 = first arg
p2 = second arg
...

Static Method

p0 = first arg
p1 = second arg
p2 = third arg
...

Return Types

return v0 → int/boolean/etc.
return-object v0 → object/reference
return-wide v0 → long/double
return-void → void

EXAMPLE – FOLLOWING A VALUE

```
const-string v0, "https://api.example.com/"  
move-object v1, v0  
invoke-static (v1), Lcom/example/Network;.>connect(Ljava/lang/String;)V
```

"https://api.example.com/"

v0

v1

connect()

Follow the value. Find where it comes from, where it goes, and what it affects.

5. REGISTER TYPE IN ACTION

```
.method public getToken()Ljava/lang/String;  
.locals 1  
sget-object v0, Lcom/example/Session;.>token:Ljava/lang/String;  
return-object v0  
.end method
```

Decoded:

Return type: String
v0: String reference
Source: static field (Session.token)

The type tells you how to interpret the value in the register.

8. PRACTICAL EXERCISE

- Pick a Small method.
- Create a register map.

Register	What it holds
p0	this / first parameter
p1	second parameter
v0	?
v1	?
v2	?

- Identify types using:
 - descriptors
 - instructions
 - method signatures
 - field types
- Trace an interesting value through the method.

BUG HUNTER PERSPECTIVE

Follow the data flow:

- Where is it created?
- Which register holds it?
- Where does it go?
- Does it enter a field?
- Does it reach a method?
- Does it influence a decision?

The register instruction itself isn't the vulnerability. It's the trail of data it carries.

Common Data Flow Examples

- Token → Network request
- URL → WebView / Intent
- User object → Authorization check
- Config flag → Debug / Feature toggle
- Input string → SQL / File / Command

6. QUICK RECAP

- Registers store values during method execution.
- Single-width values → 1 register.
- Long/double → 2 registers.
- Use move, move-object, move-wide to transfer values.
- Use move-result* to get method results.
- const / const-string load constants.
- new-instance creates objects.
- iget/put access instance fields.
- sget/sput access static fields.
- Types are determined by the instruction and context.
- Follow the data, not just the instruction.

FINAL THOUGHT

At first, registers look like random boxes.

Then you realize:

v0 → data
v1 → string
v2 → object

and suddenly, Small stops being gibberish. Because you're no longer just reading code... you're following data.

By - bithowl

In the previous chapter, we learned what can live inside a register.

We saw:

v0 → int

v1 → String reference

v2/v3 → long

v4 → object

v5 → array

But there is a new question.

How does data actually move around?

Imagine you're investigating a suspicious Android application.

You find this:

```
String token = "SECRET_TOKEN";
```

Easy.

But the real application might not look like that.

Instead, Smali might show:

```
const-string v0, "SECRET_TOKEN"
```

```
move-object v1, v0
```

```
invoke-static {v1}, Lcom/example/Auth;->send(Ljava/lang/String;)V
```

Now you need to answer:

Where did the token come from?

Which register contains it?

Where does it go next?

Does it reach the network?

Does it reach an authentication decision?

This is where **register instructions** become extremely important.

The Story - The Warehouse Conveyor Belt

Imagine an enormous warehouse.

There are thousands of boxes.

Each box has a small number: BOX 0, BOX 1, BOX 2, BOX 3, BOX 4

These are your registers. Now imagine workers moving boxes around.

One worker says: "Take whatever is in Box 0 and put it in Box 1."

That's: `move v1, v0`

Another worker says: "Put this new object into Box 2."

That's: `new-instance v2, Lcom/example/User;`

Another says: "Take the value from this object's field and put it into Box 3."

That's: `iget v3, v2, Lcom/example/User;->id:I`

And another says: "Take Box 3 and put its value back into the object's field."

That's: `iput v3, v2, Lcom/example/User;->id:I`

So Smali instructions are basically the **warehouse workers**.

Registers are the **boxes**.

Your job as a bug hunter?

Follow the box.

What Are Register Instructions?

Register instructions manipulate the values stored in registers.

They allow Android code to:

- copy values
- create objects
- load constants
- load strings
- retrieve fields
- modify fields
- receive method results
- move objects
- move wide values

The important idea is:

Data → Register → Instruction → Register → Method → Register

Once you understand this flow, Smali becomes much easier to read.

1. move

Let's start with the simplest instruction.

`move v1, v0`

Meaning: Copy the value from v0 into v1.

For example:

```
const v0, 0x2A
```

```
move v1, v0
```

Conceptually:

Before:

v0 = 42

v1 = ?

After:

v0 = 42

v1 = 42

Notice something important. move **copies** the value. It does not empty v0.

So:

v0 → 42

v1 → 42

Both registers now contain the value.

move Is Your First Data-Flow Clue

Suppose you discover:

```
const v0, 0x1
```

```
move v1, v0
```

```
move v2, v1
```

```
move v3, v2
```

A beginner might see four unrelated instructions.

A bug hunter sees: 42?

Actually, here:

v0 = 1

↓

v1 = 1

↓

v2 = 1

↓

v3 = 1

The value is travelling through registers.

This is called **data flow**.

2. move-object

Objects use: move-object

Example: move-object v1, v0

Meaning: Copy the object reference in v0 into v1.

For example:

const-string v0, "hello"

move-object v1, v0

Now:

v0 → "hello"

v1 → "hello"

Technically, for objects and strings, the register holds a **reference**, not the entire object.

That's an important distinction.

move vs move-object

Think:

move

↓

normal 32-bit value

move-object

↓

object/reference

For example:

move v1, v0

might move an integer.

While: move-object v1, v0

might move:

String

User

Activity

Context

Array

Exception

and other reference types.

3. move-wide

Now we reach wide values. Remember Chapter 10? long and double use **two register slots**.

Example:

```
const-wide v0, 0x1122334455667788
```

The value occupies:

v0 + v1

To move it:

```
move-wide v2, v0
```

Conceptually:

Before:

v0/v1 → long value

After:

v0/v1 → long value

v2/v3 → same long value

That's why you don't use normal move for wide values.

4. move-result

This one is extremely important.

Imagine Java:

```
int result = calculate();
```

Smali might look like:

```
invoke-static {}, Lcom/example/Calculator;->calculate()I
```

move-result v0

The first instruction calls the method.

The second retrieves its result.

Think:

method()

↓

result

↓

v0

So:

move-result v0

means:

Take the result returned by the previous method invocation and place it in v0.

The Important Rule

move-result normally follows an invocation.

For example:

invoke-static {}, Lcom/example/Auth;->check()Z

move-result v0

The method returns:

boolean

because:

()Z

means:

() → boolean

So now:

v0 = true/false

This becomes extremely interesting for bug hunting.

Security Decision Example

Suppose you see:

invoke-static {p0}, Lcom/example/Auth;->isLoggedIn(Landroid/content/Context;)Z

move-result v0

if-eqz v0, :not_logged_in

Follow the data:

isLoggedIn()



move-result



v0



if-eqz



security decision

That's exactly the kind of flow a bug hunter wants to identify.

5. move-result-object

What if the method returns an object?

Java:

```
String token = getToken();
```

Smali:

```
invoke-static {}, Lcom/example/Auth;->getToken()Ljava/lang/String;
```

move-result-object v0

Here:

getToken()



String reference



v0

Because the return type is:

```
Ljava/lang/String;
```

we use:

move-result-object

Example: Token Tracking

Imagine:

```
invoke-static {}, Lcom/example/Session;->getToken()Ljava/lang/String;
```

move-result-object v0

```
invoke-static {v0}, Lcom/example/Network;->send(Ljava/lang/String;)V
```

Your eyes should immediately notice:

getToken()

↓

v0

↓

send(v0)

You've discovered a potential **token data flow**.

That's far more useful than simply reading individual instructions.

6. move-result-wide

What if the method returns a long or double?

Java:

```
long timestamp = getTimestamp();
```

Smali:

```
invoke-static {}, Lcom/example/Time;->getTimestamp()J
```

move-result-wide v0

The J tells us:

long

Therefore:

move-result-wide

is used.

The result occupies:

$v0 + v1$

7. const

Now let's create values.

Example:

```
const v0, 0x1
```

This places a constant integer value into v0.

Conceptually:

$v0 = 1$

Another example:

```
const v0, 0x0
```

means:

$v0 = 0$

You will see these everywhere.

Security Flags Hide Here

Imagine:

```
const v0, 0x1
```

followed by:

```
if-eqz v0, :disabled
```

The register contains:

1

which may represent:

enabled

true

allowed

authenticated

debug mode

feature enabled

But don't assume the meaning from 1 alone. You need to trace where the value is used.

That's an important reverse-engineering habit:

Don't guess what a value means. Follow its data flow and context.

8. const-string

This is one of the most interesting instructions for Android bug hunters.

Example:

```
const-string v0, "admin"
```

Now:

v0 → "admin"

Another:

```
const-string v1, "https://api.example.com"
```

Now:

v1 → URL

Another:

```
const-string v2, "Authorization"
```

Now:

v2 → HTTP header name

Why Bug Hunters Love const-string

Because strings often reveal application behaviour.

Search for:

http

https

api

token

password

secret

Authorization

Bearer

admin

debug

internal

You may discover:

```
const-string v0, "https://api.example.com"
```

```
const-string v1, "Authorization"
```

```
const-string v2, "Bearer "
```

Then follow those registers.

For example:

```
const-string v0, "Bearer "
```

```
invoke-static {}, Lcom/example/Session;->getToken()Ljava/lang/String;
```

```
move-result-object v1
```

Now:

v0 → "Bearer "

v1 → token

The next instructions may combine them.

That's where the investigation gets interesting.

9. new-instance

Now let's create an object.

Java:

```
User user = new User();
```

Smali:

```
new-instance v0, Lcom/example/User;
```

Conceptually:

v0 → new User object

Usually you'll then see a constructor invocation:

```
new-instance v0, Lcom/example/User;
```

```
invoke-direct {v0}, Lcom/example/User;-><init>()V
```

Think:

```
new-instance
```

↓

object created

↓

constructor

↓

object initialized

Why new-instance Matters

When reverse engineering an app, object creation tells you where important structures are born.

You may see:

```
new-instance v0, Lcom/example/ApiClient;
```

or:

```
new-instance v1, Lcom/example/User;
```

or:

```
new-instance v2, Ljava/net/URL;
```

These are clues about what the application is doing.

10. iget / iput

Now we access object fields.

Suppose Java contains:

```
String token = user.token;
```

Smali might look like:

```
iget-object v0, v1, Lcom/example/User;->token:Ljava/lang/String;
```

Break it down:

iget-object

↓

read an object field

v0

↓

destination register

v1

↓

object register

User;->token

↓

field being read

So:

v1.user.token

↓

v0

iget = Read Instance Field

Think:

Object

↓

field

↓

register

Example:

```
iget v0, v1, Lcom/example/User; -> id:I
```

Means approximately:

```
v0 = v1.id;
```

For an object field, you'll commonly encounter the object form:

```
iget-object
```

For wide fields, corresponding wide forms are used.

iput = Write Instance Field

Now reverse the direction.

Java:

```
user.token = token;
```

Smali:

```
iput-object v0, v1, Lcom/example/User; -> token:Ljava/lang/String;
```

Think:

register

↓

object field

So:

v0 → token

↓

User.token

This Is Extremely Useful for Data Tracking

Suppose:

```
const-string v0, "SECRET"
```

```
iput-object v0, v1, Lcom/example/User;->token:Ljava/lang/String;
```

You can trace:

"SECRET"

↓

v0

↓

User.token

Later:

```
iget-object v2, v1, Lcom/example/User;->token:Ljava/lang/String;
```

Now:

User.token

↓

v2

The value has travelled:

const-string

↓

v0

↓

iput-object

↓

User.token

↓

iget-object

↓

v2

That's the kind of chain you want to build during reverse engineering.

11. sget / sput

What if the field is static?

Java:

```
String token = Session.token;
```

Smali:

```
sget-object v0, Lcom/example/Session;->token:Ljava/lang/String;
```

This means:

Read a static field.

sget = Read Static Field

Think:

Static Field



Register

Example:

```
sget v0, Lcom/example/Config;->DEBUG:I
```

Conceptually:

```
v0 = Config.DEBUG;
```

sput = Write Static Field

Example:

```
sput-object v0, Lcom/example/Session;->token:Ljava/lang/String;
```

Conceptually:

```
Session.token = v0;
```

Now the flow becomes:

v0



Session.token

Instance vs Static Fields

Remember this simple rule:

Instance field

Belongs to an object:

iget

iput

Conceptually:

user.token

Static field

Belongs to a class:

sget

sput

Conceptually:

Session.token

A simple mental model:

INSTANCE

Object

|

└─ field

STATIC

Class

|

└─ field

The Real Skill: Follow the Data

Knowing individual instructions is useful.

But memorizing instructions isn't the real goal.

The real goal is:

DATA-FLOW ANALYSIS

Suppose you find:

const-string v0, "https://api.example.com"

move-object v1, v0

invoke-static {v1}, Lcom/example/Network;->connect(Ljava/lang/String;)V

Don't read it as three separate lines.

Read it as:

URL

↓

v0

↓

v1

↓

connect()

That's the mindset.

Bug Hunter Example #1 — Password

Suppose you find:

const-string v0, "password123"

move-object v1, v0

invoke-static {v1}, Lcom/example/Auth;->login(Ljava/lang/String;)V

Your data flow is:

"password123"

↓

v0

↓

v1

↓

login()

Now investigate:

- Is this a test credential?
- Is it production?

- Is it sent over the network?
- Is it used for authentication?
- Is it exposed in logs?
- Is it only a dummy value?

Don't immediately call every hardcoded string a vulnerability.

Trace it first.

#Bug Hunter Example #2 — API Token

Imagine:

```
sget-object v0, Lcom/example/Session;->token:Ljava/lang/String;
```

```
move-object v1, v0
```

```
invoke-static {v1}, Lcom/example/Api;->request(Ljava/lang/String;)V
```

Follow it:

Session.token

↓

v0

↓

v1

↓

Api.request()

Now you know:

A static token field reaches an API request.

That's a meaningful lead.

Bug Hunter Example #3 — URL

You find:

```
const-string v0, "https://internal.example.com"
```

```
invoke-static {v0}, Lcom/example/Api;->open(Ljava/lang/String;)V
```

Flow:

internal URL

↓

v0

↓

open()

Now investigate:

Who controls the URL?

Can it be modified?

Is it trusted?

Is it used for redirects?

Is it used for API communication?

Is certificate validation performed?

The register instruction itself isn't necessarily the vulnerability.

It is the **trailhead**.

Bug Hunter Example #4 — Security Flag

Consider:

sget v0, Lcom/example/Config;->DEBUG:I

if-eqz v0, :normal

invoke-static {}, Lcom/example/Debug;->enable()V

Data flow:

Config.DEBUG

↓

v0

↓

condition

↓

security-sensitive branch

Now investigate where:

Config.DEBUG

is written.

You might find:

```
const v1, 0x1
```

```
sput v1, Lcom/example/Config;->DEBUG:I
```

Now you've connected:

```
1
```

```
↓
```

```
v1
```

```
↓
```

```
Config.DEBUG
```

```
↓
```

```
v0
```

```
↓
```

```
branch
```

```
↓
```

```
debug functionality
```

This is how a few register instructions can reveal an entire execution path.

The Register Tracking Technique

When analysing Smali, make a tiny table.

For example:

Register Value

v0	API URL
----	---------

v1	token
----	-------

v2	authentication result
----	-----------------------

v3	User object
----	-------------

Then update it as you read.

Example:

```
const-string v0, "https://api.example.com"
```

You write:

```
v0 = API URL
```

Then:

move-object v1, v0

Update:

v1 = API URL

Then:

invoke-static {v1}, Lcom/example/Api;->connect(Ljava/lang/String;)V

Now you know:

API URL → connect()

This simple technique becomes incredibly powerful with larger methods.

Practice — Objects

Java:

User user = new User();

Smali:

new-instance v0, Lcom/example/User;

invoke-direct {v0}, Lcom/example/User;-><init>()V

Register map:

v0 → User object

Practice — Field Access

Java:

String token = user.token;

Smali:

iget-object v1, v0, Lcom/example/User;->token:Ljava/lang/String;

Register map:

v0 → User object

User.token

↓

v1

So:

v1 = user.token

Practice — Method Result

Java:

```
boolean authenticated = checkAuth();
```

Smali:

```
invoke-static {}, Lcom/example/Auth;->checkAuth()Z
```

```
move-result v0
```

Register map:

v0 → authenticated

Practice — Put Everything Together

Java:

```
String token = getToken();
```

```
send(token);
```

Smali:

```
invoke-static {}, Lcom/example/Auth;->getToken()Ljava/lang/String;
```

```
move-result-object v0
```

```
invoke-static {v0}, Lcom/example/Api;->send(Ljava/lang/String;)V
```

Data flow:

```
getToken()
```

↓

v0

↓

```
send()
```

This is the exact mental model you should develop.

Quick Reference

Instruction	Purpose
move	Move normal value
move-object	Move object/reference
move-wide	Move wide value

Instruction	Purpose
move-result	Receive normal method result
move-result-object	Receive object result
move-result-wide	Receive wide result
const	Load integer constant
const-string	Load string reference
new-instance	Create object
iget	Read instance field
iput	Write instance field
sget	Read static field
sput	Write static field

And remember the basic direction:

move

$A \rightarrow B$

iget

object.field $\rightarrow$ register

iput

register $\rightarrow$ object.field

sget

Class.field $\rightarrow$ register

sput

register $\rightarrow$ Class.field

move-result

method() → register

new-instance

register ← new Object

Bug Hunter Checklist

When you encounter an interesting value, ask:

1. Where was it created?

Look for:

const

const-string

new-instance

move-result

2. Which register contains it?

Example:

v0

3. Where does the register go?

Look for:

move

move-object

invoke-*

iput

sput

4. Does it enter an object field?

Look for:

iput

iput-object

5. Does it come from a field?

Look for:

iget

iget-object

sget

sget-object

6. Does it reach a method?

Look for:

invoke-*

7. Does it influence a security decision?

Look for:

if-*

if-eqz

if-nez

packed-switch

sparse-switch

Those control-flow instructions will become much more important in the next chapters.

Quick Recap

You now know how Smali moves data.

move

moves normal values.

move-object

moves references.

move-wide

moves wide values.

move-result

captures normal method results.

move-result-object

captures object results.

move-result-wide

captures wide results.

const

creates numeric constants.

const-string

loads strings.

new-instance

creates objects.

iget / iput

read and write instance fields.

sget / sput

read and write static fields.

But the most important lesson isn't any individual instruction.

It's this: **Follow the data.**

Final Thought

A Smalltalk method can look terrifying at first.

Hundreds of registers.

Dozens of instructions.

Strange descriptors.

Labels everywhere.

But don't try to understand everything at once.

Pick one interesting value.

Maybe: password

Maybe: token

Maybe: API URL

Maybe: isAdmin

Then follow it.

Where was it created?

↓

Which register holds it?

↓

Where is it moved?

↓

Which field stores it?

↓

Which method receives it?

↓

Does it influence a security decision?

That is the beginning of real Smali analysis.

Because when you're hunting bugs, you don't need to understand **every box in the warehouse**.

You just need to know:

Which box contains the valuable package and where that package is going.

Next chapter: Control Flow in Smali **“Which Door Does the Code Take?”**

BY – bithowl